AI INFRA RESEARCH SERIES

HiLS-Attention

从块选择的数学依据，到长上下文与 GPU 实现

分析对象

Hierarchical Sparse Attention Done Right: Toward Infinite Context Modeling

作者与机构

Xiang Hu、Xinyu Wei、Hao Gu、Minshen Zhang 等；腾讯混元、上海科技大学、香港科技大学、加州大学圣迭戈分校。

版本范围

arXiv:2607.02980v1，2026-07-03；结合官方 HiLS-Attention 仓库与其 SGLang 推理后端说明。

核心判断

HiLS 的关键不是简单少看一些 token，而是让块级检索分数真正参与注意力输出，使语言模型损失能够训练检索表示。它改造的是注意力机制；并不自动解决全部 KV 存储，也不等于严格的无限上下文。

阅读定位：机制与公式、系统实现、实验口径、局限与复现。

说明：性能数据均标明来源；本报告没有独立训练模型或复跑 GPU 性能。推导与教学算例不作为作者实测结果。

整理日期：2026 年 10 月 31 日

阅读导航

1 先确定它处在哪一层 2

2 为什么“每块平均一下”不够 2

3 核心推导：摘要 key 加上熵 3

4 为什么分层 softmax 能训练检索器 4

5 Query Calibration、位置编码与 GQA 4

6 GPU 角度：少算之外，还要算得成块 5

7 实验结果该怎么读 6

8 从论文到可运行服务：当前边界 7

9 值得保留的结论与仍需验证的问题 7

10 建议的最小复现与研究切口 7

11 原始资料与阅读顺序 8

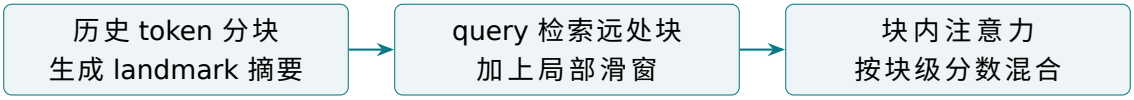
文中的 [S1]、[S2] 等编号指向文末原始资料。引用附带章节、表号或源码位置，便于核对；不同报告的编号独立。

1 先确定它处在哪一层

HiLS-Attention 中的 HiLS 是 **Hierarchical Landmark Sparse Attention**。它属于模型结构研究：把完整上下文切为块，用可学习的 landmark 表示检索远处的块，再在选中的块内部做细粒度注意力。最终仍然输出每个 query 的注意力结果。[S1，§2–3]

它不是 HiSparse。HiSparse 接收既有稀疏选择器给出的索引，管理这些索引对应的 KV 在 CPU/GPU 之间的驻留；HiLS 决定索引如何选择、不同块如何加权。二者在概念上可以组合，但这需要对齐 chunk、KV 格式和运行时协议，不能据名字相近认定已经集成。

它也不是 KDA。KDA 用有限大小的递归状态压缩历史；HiLS 仍保留对远处原始 KV 的寻址能力，只是每次读取其中一部分。



阅读主线：先理解“应该给每个块多少注意力质量”，再理解“如何廉价估计它”，最后理解“估计器如何学到任务相关的选择”。

2 为什么“每块平均一下”不够

2.1 完整注意力诱导出的块质量

为避免符号混淆，本报告用 S 表示每块 token 数，K 表示选择的远处块数，d 表示 head 维度。令

$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d}}, \qquad Z_{ic} = \sum_{j \in c} \exp(s_{ij}).$$

Z_{ic} 是 query i 分给块 c 的未归一化质量。若能知道所有块的 Z_{ic}，按它们选择 top-K 块就很自然。然而精确计算这些量仍要查看全部 token 的 QK 分数，失去了筛选应当带来的计算节省。论文把这种方法作为 **naive Block Sparse Attention 的参照**，而不是高效实现本身。[S1，§2.1]

一个常见近似是用均值 key:

$$\bar{k}_c = \frac{1}{S} \sum_{j \in c} k_j, \qquad q_i^T \bar{k}_c / \sqrt{d} = \text{mean}_{j \in c}\, s_{ij}.$$

问题在于，需要近似的是 **LogSumExp**，不是平均 logit。大量较低分和一个极高分，可能具有很高的总注意力质量，却被均值冲淡。

2.2 一个可以手算的教学例子

设两个块各有四个分数：块 A 为 $(4, 0, 0, 0)$，块 B 为 $(1.5, 1.5, 1.5, 1.5)$。均值分别为 1 和 1.5，因此均值摘要会偏向 B。但真实质量为

$$Z_A = e^4 + 3 \approx 57.60, \qquad Z_B = 4e^{1.5} \approx 17.93.$$

A 里那个很相关的 token 足以使真实质量超过 B 三倍。这个例子不是论文实验，只解释为什么“平均语义相似度”与“对答案有用的注意力质量”可能不同。仅换成最大值也不是通用解，因为它会忽视多个中等相关 token 的累积贡献。

3 核心推导：摘要 key 加上熵

3.1 从 LogSumExp 的一阶展开出发

对固定块定义

$$f_c(q) = \log \sum_{j \in c} \exp(q^T k_j / \sqrt{d}).$$

在 landmark query q'_c 附近展开。先计算

$$p_j = \mathrm{softmax}_{j \in c} \left((q'_c)^T k_j / \sqrt{d} \right),$$

再令

$$k'_c = \sum_{j \in c} p_j k_j, \qquad b'_c = -\sum_{j \in c} p_j \log p_j.$$

因为 $\nabla_q f_c(q'_c) = k'_c / \sqrt{d}$，一阶展开可以整理为

$$\hat{s}_{ic} = \frac{q_i^T k'_c}{\sqrt{d}} + b'_c, \qquad \hat{Z}_{ic} = \exp(\hat{s}_{ic}).$$

这是论文 Proposition 3.1 的关键结构。[S1，式 7–8 及附录 B] 第一项表达 query 与块摘要的匹配，第二项是摘要分布的熵：分数均匀时接近 $\log S$，单个 token 主导时接近 0。这个偏置不是任意加的常数，而是从展开式中自然出现。

生成摘要可理解为用 q'_c 对块内的 K 做一次以 K 为 value 的注意力。实际方法把一个 landmark token 放到块末尾，用它的 query 形成摘要；因果边界保证完整块先形成，再供后续 query 检索。[S1，§3]

3.2 更进一步：近似误差恰好是一个 KL 散度

下面是依据上述定义作出的**补充推导**，不是论文新增实验。若 p 固定为 landmark 诱导的分布，并定义真实 query 对块内的分布 $r_q(j)$，则

$$\begin{aligned} D_{\mathrm{KL}}(p \| r_q) &= \sum_j p_j \log p_j - \sum_j p_j \left(q^T k_j / \sqrt{d} - f_c(q) \right) \\ &= f_c(q) - \left(q^T k'_c / \sqrt{d} + H(p) \right). \end{aligned}$$

所以，在相同 key 和 query 定义下，线性代理是一个下界；当真实 query 诱导的块内分布恰好等于

landmark 分布时，误差为零。这提供了比“可学习摘要更强”更具体的理解：**一个摘要只能很好覆盖某些 query 分布，训练要让它覆盖更有用的区域**。

不能进一步宣称单个摘要能无损表示所有 query。Query Calibration 改变检索 query 后，也不能不加条件地把上面的下界套到原始 query 的质量上。

4 为什么分层 softmax 能训练检索器

4.1 不再在 top-K 后丢掉检索分数

很多稀疏方法用摘要选出块，随后丢掉摘要分数，只在选中的原始 KV 上执行普通注意力。离散索引不会自动提供有用的连续梯度路径。HiLS 则让块级代理分数直接控制最终输出中的块权重。[S1, §3]

令 $\mathcal{I}_i$ 为选中的远处块，局部滑窗的精确质量为 $Z_{i,w}$。归一化分母为

$$\hat{\mathcal{Z}}_i = Z_{i,w} + \sum_{c \in \mathcal{I}_i} \hat{Z}_{ic}.$$

对远处块内部 token：

$$w_{ij} = \underbrace{\frac{\hat{Z}_{ic}}{\hat{\mathcal{Z}}_i}}_{\text{块间权重}} \underbrace{\frac{e^{s_{ij}}}{Z_{ic}}}_{\text{块内权重}}, \quad j \in c.$$

块内部先提取一个内容摘要，再由块级分数决定该摘要在总输出中的贡献。局部滑窗采用相应的精确质量。若把所有代理质量换回精确质量，上式就回到选中集合上的普通 softmax。

因此，语言模型损失可以通过最终混合权重，更新摘要 key、landmark 和校准参数。这是方法中最重要的梯度设计，而不仅是一个检索 heuristic。

4.2 “端到端训练”不意味着硬 top-K 可微

必须划清边界：硬 top-K 选中的离散索引仍然是离散的。HiLS 提供了经过所选块权重的可微路径，不是对所有未选块都获得了无偏、平滑的选择梯度。训练早期选择质量、局部滑窗的保障、初始化和 continued pretraining 都可能影响它能否学到好摘要。

一个合理的审视问题是：遇到很少被选中的块，其摘要如何获得足够训练信号？论文给出了整体实验支持，但不能把“端到端”解释成消除了离散检索的一切学习难题。

5 Query Calibration、位置编码与 GQA

5.1 检索 query 和读取 query 不必完全相同

论文加入低秩残差校准，把适合 token 级读取的 query 调整成更适合块级质量估计的 query：

$$\hat{q}_i = q_i + W^{\text{up}} W^{\text{down}} h_i.$$

新增参数相对很少，但作用不是增加普通 FFN 容量，而是校准 “用哪个问题去询问块摘要”。理解这个模块时，应把完整块内注意力与块级检索两个空间分开。[S1，§3.1]

5.2 HoPE 的作用不能被归给稀疏选择

HiLS 的长上下文结果包含位置编码设计。HoPE 保留训练长度能够覆盖周期的 RoPE 维度，对过长周期的部分采用无位置编码处理，以减轻长度外推问题。[S1，§3.1]

所以，“从 8K 外推到很长上下文” 来自机制、位置处理和训练共同作用。严格归因需要固定其他条件，比较有无 HoPE、有无校准、有无 landmark、相同预算下的块选择，而不能只比较最终整套配置。

5.3 GQA 组内共享选择

组查询注意力中多个 Q head 共享 KV。若每个 head 独立读取一组完全不同的块，KV 复用会被破坏。论文在组内组合归一化后的块重要性，形成共享选择，使内容表达和硬件复用之间取得折中。[S1，§3.1]

共享选择会扩大某些 head 实际访问的集合，也可能使单个 head 的最佳选择被组策略影响。因此它既是模型实现设计，也是内存访问设计。

6 GPU 角度：少算之外，还要算得成块

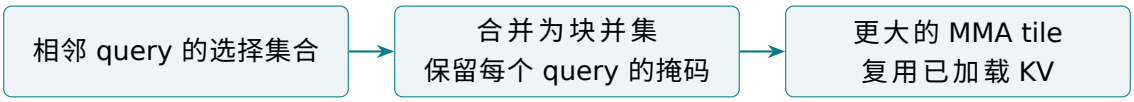
传统逐 query 的稀疏路径容易出现很小的 Tensor Core 操作。若 G 是 GQA 组内 query head 数，逐 query 计算近似为

$$(G, d) \times (d, S).$$

HiLS 将相邻的 M 个 query 打包，取它们已选块的并集，形成

$$(MG, d) \times (d, S).$$

相邻 query 的选择通常具有重叠，因此可以复用 KV；对不属于某个 query 选择的块施加掩码，保持其语义。论文 Fig.4 展示了这种从 query 轴与 head 轴共同构造矩阵块的方式。[S1，§3.2、Fig.4]



收益取决于选择重叠。若相邻 query 的选块几乎完全不同，并集膨胀会引入大量无效计算；因此 “增加 M” 不可能永远有利。合理的 kernel 调优指标应包含 union size、有效元素比例、实际 HBM 字节和寄存器占用，而不只是理论稀疏率。

6.1 复杂度的严格口径

假设每个 query 仍需对全部 N/S 个摘要打分，训练中的总体工作量粗略为

$$O(N^2d/S) + O(NKSd) + O(NWd),$$

另有生成摘要的线性开销。这个估算忽略 head 数及具体常数，用于强调：**块摘要降低了全局扫描的粒度，并未让所有操作都与上下文长度无关**。

同样，完整历史 KV 可能仍需要存储。每步只激活 $KS + W$ 个 token，不等于把全部历史 KV 都删除了。这正是它与 HiSparse 类存储层技术可以互补的原因。

7 实验结果该怎么读

7.1 小模型实验：外推任务之间差异很大

345M 模型以 8K 训练长度进行比较；稀疏预算为远处 2K token 加局部 512 token。论文还引入额外 MLP 的完整注意力基线，控制 HiLS 新增参数的影响。[S1，§5]

方法／配置	8K PPL	32K PPL	128K PPL	512K PPL
Full Attention + HoPE	4.95	6.42	>100	—
NSA + RoPE	5.01	7.62	11.75	19.14
HiLS + HoPE	4.94	4.34	4.71	5.95

数据取自论文 Table 1；“—” 在这里表示本表未列，不推断具体失败原因。PPL 应在相同样本与截取口径下比较，跨长度的变化不能直接解释为模型越来越聪明。

更值得注意的是 Table 2 的任务分化：在 4M 长度下，345M HiLS 的单针检索为 **96**，多键多问为 **89**，变量追踪为 **43**。所以 “512 倍长度外推且超过 90%” 描述的是特定检索任务，绝不是所有推理任务都超过 90%。[S1，Table 2]

7.2 7B 实验：CPT 与轻量调参是两条不同路径

7B OLMo3 的主要结果使用 **50B tokens continued pretraining**。Fig.1 的 RULER 平均分从 8K 的 99.0，到 128K 的 94.7、512K 的 89.3、1M 的 81.7。LongBench 的总体展示值为 28.0→33.2，短任务集合均分约 43.2→43.4。[S1，Fig.1、§6]

另一条路径是冻结大部分基础模型，仅训练 landmark 和校准参数，参数不到 1%，使用不超过 5B tokens。它与上面的 50B 全参数 CPT 不能混为 “只训练不到 1% 参数就得到所有展示结果”。[S1，§4]

图中的长序列速度提升也应按长度、batch、模型和实现比较。后端 README 明确把约 13.5 倍 prefill、15.7 倍 decode 对应到 345M 模型、512K 上下文；不能把这些数字与上面的 7B 质量实验混为同一个实验，也不应推广为所有线上请求的统一加速比。[S3]

8 从论文到可运行服务：当前边界

官方仓库公开训练、评测和 checkpoint 转换流程，并把 SGLang 后端指向独立 fork `alexzms/SGLang-HiLS`。文档要求 page size 等于 chunk size 64，并说明当前后端采用单序列 prefill、关闭 overlap scheduler。[S2]

该 fork 还明确定位为速度基准，而非完全对齐的生产服务：参考实现会在 SWA 层屏蔽插入的 landmark tokens，而后端暂未复现这一行为。因此“接近参考模型”不能写成逐位一致或完全等价，速度比较之外还需要单独核对输出与质量。[S3]

这些是现实约束。单请求长上下文 kernel 很快，不代表多租户连续批处理下已经达到最佳吞吐。复现应先比较同 batch、相同输出、相同页尺寸的性能，再测引擎级请求到达、排队和 prefill/decode 交替。

此外，算法必须依靠对应训练或 CPT 后的权重。直接在任意 dense 模型上将普通注意力替换为 HiLS，不能期待自然保持质量。

9 值得保留的结论与仍需验证的问题

最强贡献是把“块选择分数”从一次性过滤信号变成最终注意力函数的组成部分，并给出有数学来源的摘要结构。这比任意 pooling 加 top-K 更可解释。

仍需谨慎的地方包括：有限摘要容量、离散选择的训练覆盖、长序列摘要扫描、GQA 共享选择的损失，以及极长上下文任务的代表性。RULER 检索、变量追踪、真实代码仓库理解和多文档推理不是同一能力。

本报告的判断：HiLS 是模型机制与 kernel 共同设计的好例子，但标题中的“infinite context”是方向目标，不是有限内存机器上无界且无损记忆的证明。

10 建议的最小复现与研究切口

先实现小规模完整注意力、naive BSA 和 HiLS 的 FP32 参考，在固定选块条件下验证分层公式、归一化和梯度；随后加入硬选择，分别测试局部窗口、chunk 末尾、空历史和边界 causal mask。这里先验证数学，而不是直接跑大模型。

第二步固定真实 query 轨迹，记录选块并集大小，扫描 query packing 的 M、chunk size、GQA 比例。在 HBM 字节和 Tensor Core 利用率之外，同时记录多余掩码计算比例，这有助于判断瓶颈究竟是选块、gather 还是 MMA。

第三步用同一 CPT checkpoint 比较注意力 backend，报告首 token 延迟、每 token 延迟、有效吞吐和质量。评测分成 8K 内、训练长度外、复杂推理三组，不能只看一个最长长度的单针分数。

一个值得研究但尚未证明的方向是：**根据选择集合的重叠度自适应决定 query packing 粒度**。相比固定 M，它可能在“更大 MMA”与“并集膨胀”之间找到更稳健的折中。另一个方向是与分层 KV 存储结合，同时测缓存命中率和模型质量，而不是默认稀疏计算自然意味着少占显存。

11 原始资料与阅读顺序

[S1] 原论文 v1。 arXiv 2607.02980v1。建议顺序：§2 的质量定义 → §3 与 Proposition 3.1 → Fig.4 → Tables 1–2 → §6–7。原始图表数字为作者报告。

[S2] 官方项目。Tencent-Hunyuan/HiLS-Attention。核对 README 的训练配置、CPT 两条路径与 Efficient Inference 一节; README 内容哈希 `4706bf83562a5e59d491b43b8132ab900e885e46`。

[S3] 推理后端。SGLang-HiLS README。核对开头的速度基准说明与 LMK/SWA 对齐限制；读取内容哈希 `705687a885d2caf108b8426cf479ac849b0850ed`。此项目由官方仓库指向，但不能据此宣称已与所有 SGLang 上游功能兼容。

本报告没有运行训练、SGLang 服务或自定义 kernel。KL 误差推导、复杂度拆解、教学算例与复现设计属于解释性分析。